PHB 228: Homework 1

Part 1: Vectorization and Data Structures (15 points)

1.1 Vectorization (8 points)

Using the built-in `iris` dataset:

a) (2 points) Create a function `vectorized_zscore` that standardizes a numeric vector using vectorized operations (z = (x - mean(x)) / sd(x)). Create a second function `loop_zscore` that performs the same computation using a `for` loop.

```
df <- iris
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

```
#dim(df)
#colSums(is.na(df))
```

```
vectorized_zscore <- function(x){
  z = (x - mean(x))/sd(x)
  return(z)
}

loop_zscore <- function(x){
  n <- length(x)
```

```
  # calc mean
  sum_x <- 0
  for (i in seq_len(n)){
    sum_x <- sum_x + x[i]
  }
  mean_x <- sum_x/n

  # calc sd
  sum_sd <- 0
  for (i in seq_len(n)){
    sum_sd <- sum_sd + (x[i] - mean_x)^2
  }
  sd_x <- sqrt(sum_sd/(n - 1))

  z <- numeric(n) # pre allocate
  for (i in seq_len(n)){
    z[i] <- (x[i] - mean_x)/sd_x
  }

  return(z)
}
```

```
x <- df$Sepal.Length
vectorized_z <- vectorized_zscore(x)
loop_z <- loop_zscore(x)

#head(vectorized)
#head(loop_z)
all.equal(vectorized_z, loop_z)
```

```
[1] TRUE
```

b) (3 points) Use `bench::mark()` to compare performance with at least 50 iterations. Create a bar plot comparing median execution times.

```
library(bench)
library(ggplot2)
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
bm <- bench::mark(
  vectorized = vectorized_zscore(x),
  loop = loop_zscore(x),
  iterations = 50
)

bm
```

```
# A tibble: 2 x 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 vectorized   9.76us   10.7us    89776.   21.91KB        0
2 loop        23.08us   23.6us    42071.    1.22KB        0
```

```
#str(bm)
bm$expression <- as.character(bm$expression)

ggplot(data = bm, mapping = aes(x = expression, y = median, fill = expression)) +
  geom_col() +
  labs(title = "Median execution time: vectorized vs. loop",
       x = "Implementation",
       y = "Median time")
```

c) (3 points) Using the `ChickWeight` dataset, create a `Growth.Rate` column (weight change per day for each chick) and a `Weight.Gain.Percent` column (percentage gain from first to last measurement). Calculate average growth rate and mean maximum weight by diet group.

```
df1 <- ChickWeight
head(ChickWeight)
```

```
  weight Time Chick Diet
1     42    0     1    1
2     51    2     1    1
3     59    4     1    1
4     64    6     1    1
5     76    8     1    1
6     93   10     1    1
```

```
#str(ChickWeight)
```

```
chick_gw <- df1 |>
  group_by(Chick) |>
  mutate(
    Growth.Rate = (weight - first(weight)) / (Time - first(Time)),
```

```
    Weight.Gain.Percent = (last(weight) - first(weight)) / first(weight) * 100
  ) |>
  ungroup()

chick_summary <- chick_gw |>
  group_by(Chick, Diet) |>
  summarise(
    chick_ag = mean(Growth.Rate, na.rm = TRUE),
    chick_mw = max(weight),
    .groups = "drop"
  )

diet_summary <- chick_summary |>
  group_by(Diet) |>
  summarise(
    avg_gr = mean(chick_ag),
    mean_mw = mean(chick_mw),
    .groups = "drop"
  )
```

```
diet_summary
```

```
# A tibble: 4 x 3
  Diet  avg_gr mean_mw
  <fct>  <dbl>   <dbl>
1 1       4.77    158.
2 2       6.84    216.
3 3       8.27    272.
4 4       8.10    230.
```

1.2 Data Structures and Functional Programming (7 points)

Using the `palmerpenguins` dataset:

a) (2 points) Extract the unique species and create a list where each element contains data for one species. Add a sample size attribute to each element.

```
library(palmerpenguins)
```

```
Attaching package: 'palmerpenguins'
```

```
The following objects are masked from 'package:datasets':

    penguins, penguins_raw
```

```r
unique(penguins$species)
```

```
[1] Adelie    Gentoo    Chinstrap
Levels: Adelie Chinstrap Gentoo
```

```r
penguin_species <- split(penguins, penguins$species)

penguin_species <- lapply(penguin_species, function(df){
  attr(df, "sample_size") <- nrow(df)
  df
})
```

```r
# check
sapply(penguin_species, function(df) attr(df, "sample_size"))
```

```
   Adelie Chinstrap    Gentoo
      152        68       124
```

```r
table(penguins$species)
```

```
   Adelie Chinstrap    Gentoo
      152        68       124
```

b) (1 point) Use `lapply()` to calculate the mean of each numeric variable (excluding NAs). Redo using `map_dbl()`.

```r
numeric_data <- penguins[sapply(penguins, is.numeric)]

lapply(numeric_data, mean, na.rm = TRUE)
```

```
$bill_length_mm
[1] 43.92193

$bill_depth_mm
```

```
[1] 17.15117

$flipper_length_mm
[1] 200.9152

$body_mass_g
[1] 4201.754

$year
[1] 2008.029
```

```
library(purrr)

map_dbl(numeric_data, mean, na.rm = TRUE)
```

```
   bill_length_mm     bill_depth_mm flipper_length_mm       body_mass_g
         43.92193          17.15117         200.91520        4201.75439
             year
      2008.02907
```

c) (1 point) Use `tapply()` to find the mean body mass by species and by the combination of species and sex.

```
#names(penguins)

# body mass by species
tapply(penguins$body_mass_g, penguins$species, mean, na.rm = TRUE)
```

```
   Adelie Chinstrap    Gentoo
 3700.662  3733.088  5076.016
```

```
# body mass by spices and sex
tapply(penguins$body_mass_g, list(penguins$species, penguins$sex), mean, na.rm = TRUE)
```

```
            female     male
Adelie    3368.836 4043.493
Chinstrap 3527.206 3938.971
Gentoo    4679.741 5484.836
```

d) (1 point) Demonstrate R's copy-on-modify semantics: create a vector, create a reference to it, modify the reference, and explain what happens (2-3 sentences).

At first, x and y are pointing to the same object in memory, so y <- x does not actually make a copy. R uses copy on modify semantics, which means a copy only gets made once you actually change one of them. So when we change the first index value y[1] <- 6, R makes a new copy for y to modify and x stays the same. After that, x and y are two separate objects.

```
library(lobstr)
```

```
Attaching package: 'lobstr'

The following object is masked from 'package:dplyr':

    src
```

```
x <- c(1,2,3,4,5)
cat(x,"\n" ,"x address:", obj_addr(x), "\n")
```

```
1 2 3 4 5
 x address: 0x1529e4e18
```

```
y <- x
cat(y, "\n","y address:", obj_addr(y), "\n")
```

```
1 2 3 4 5
 y address: 0x1529e4e18
```

```
y[1] <- 6
cat(y,"\n" ,"y address:", obj_addr(y), "\n")
```

```
6 2 3 4 5
 y address: 0x152a1f828
```

e) (2 points) Rewrite the following inefficient code using preallocation. Compare execution times with `system.time()` and explain why your version is faster (2-3 sentences).

The original version uses c(), which combines values into a vector, then on each iteration R has to allocate a new vector and copy all the existing values over, which gets slower and slower as more elements pile up. The preallocated version creates the full sized vector once at the start, then just fills in one slot per iteration, which is why it runs in 0.003 seconds instead of 0.196.

```r
result <- numeric(0)
for (i in 1:10000) {
  result <- c(result, i^2)
}
```

```r
# original version
system.time({
  result <- numeric(0)
  for (i in 1:10000) {
    result <- c(result, i^2)
  }
})
```

```
   user  system elapsed
  0.124   0.050   0.194
```

```r
# my version
system.time({
  result2 <- numeric(10000) # pre allocate
  for (i in 1:10000){
    result2[i] <- i^2
  }
})
```

```
   user  system elapsed
  0.002   0.000   0.002
```

Part 2: Matrix Operations and Decompositions (30 points)

2.1 Matrix Operations (8 points)

Consider the following matrices:

```r
A <- matrix(c(4, 2, 1, 5), nrow = 2)
B <- matrix(c(3, 1, 0, 2), nrow = 2)
C <- matrix(c(1, 3, 5, 2, 4, 6), nrow = 2)
```

a) (4 points) Calculate and explain the operation performed in each case: `A + B`, `A * B`, `A %*% B`, `t(A) %*% C`, `solve(A)`.

```
# set up
A <- matrix(c(4, 2, 1, 5), nrow = 2)
B <- matrix(c(3, 1, 0, 2), nrow = 2)
C <- matrix(c(1, 3, 5, 2, 4, 6), nrow = 2)
```

A + B adds the corresponding entries together. Each cell (i, j) in the result is A[i, j] + B[i, j].

```
A + B
```

```
     [,1] [,2]
[1,]    7    1
[2,]    3    7
```

A * B multiples the corresponding entries together. Each cell (i, j) in the result is A[i, j] * B[i, j].

```
A * B
```

```
     [,1] [,2]
[1,]   12    0
[2,]    2   10
```

A %*% B does proper matrix multiplication, rows x columns like in linear algebra. Each cell in the result is the dot product of a row from A and a column from B.

```
A %*% B
```

```
     [,1] [,2]
[1,]   13    2
[2,]   11   10
```

t(A) %% C. *First, t(A) flips A over its diagonal, so rows become columns and columns become rows. Then we multiply the transposed matrix by C, which is 2 x 3. We can check its dimensions by (2 x 2) %*% (2 x 3) = (2 x 3).*

```
t(A) %*% C
```

```
     [,1] [,2] [,3]
[1,]   10   24   28
[2,]   16   15   34
```

Solve(A) computes the inverse of A, meaning A %*% solve(A) gives the identity matrix. This one works for square, invertible matricies.

```
solve(A)
```

```
           [,1]        [,2]
[1,]  0.2777778 -0.05555556
[2,] -0.1111111  0.22222222
```

b) (4 points) Create a function `is_symmetric` that returns TRUE if a matrix is symmetric and FALSE otherwise. Print an error if the input is not square. Test on A, B, and `t(A) %*% A`.

```
is_symmetric <- function(X){
  # check if input is square
  if (nrow(X) != ncol(X)){
    stop("Input is not square")
  }

  # check symmetry
  return(all(X == t(X)))
}

is_symmetric(A) # False
```

```
[1] FALSE
```

```
test_matrix <- matrix(c(1,2,3, 2,4,5, 3,5,6), nrow = 3)
is_symmetric((test_matrix))
```

```
[1] TRUE
```

2.2 Eigenvalue Decomposition (10 points)

```r
M <- matrix(c(4, 2, 2, 3), nrow = 2)
```

a) (3 points) Compute eigenvalues and eigenvectors using `eigen()`.

```r
M <- matrix(c(4, 2, 2, 3), nrow = 2)
#M

eig <- eigen(M)
eigenvalues <- eig$values
eigenvectors <- eig$vectors

eigenvalues
```

```
[1] 5.561553 1.438447
```

```r
eigenvectors
```

```
           [,1]       [,2]
[1,] -0.7882054  0.6154122
[2,] -0.6154122 -0.7882054
```

b) (3 points) Verify the eigenvector equation Mx = lx holds for each pair.

```r
for (i in seq_along(eig$values)){
  xi <- eig$vectors[, i]
  li <- eig$values[i]
  cat("pair", i, all.equal(as.vector(M %*% xi), li * xi), "\n")
}
```

```
pair 1 TRUE
pair 2 TRUE
```

c) (4 points) Reconstruct M using M = PDP^{-1} and verify it matches the original.

```r
# P: matrix of eigenvectors
# D: diagonal matrix of eigenvalues
# P-1: inverse of P

P <- eig$vectors
D <- diag(eig$values)
P_inverse <- solve(P)
```

```
M_reconstructed <- P %*% D %*% P_inverse
M_reconstructed
```

```
     [,1] [,2]
[1,]    4    2
[2,]    2    3
```

2.3 SVD and QR Decomposition (12 points)

a) **SVD** (6 points): Compute the SVD of the matrix below. Create a rank-1 approximation, calculate the Frobenius norm of the difference, and explain what information is lost.

Rank-1 approximation only keeps the biggest singular value (5) then drops the other two (5 and 1). It captures just one main pattern in X, but misses another important one since there were two values tied at 5, and also loses smaller detail from the 1.

```
X <- matrix(c(3, 2, 2, 2, 3, -2, 2, -2, 3), nrow = 3)
```

```
X <- matrix(c(3, 2, 2, 2, 3, -2, 2, -2, 3), nrow = 3)
#X

# compute SVD
svd_X <- svd(X)

U <- svd_X$u
D <- svd_X$d
V <- svd_X$v
D
```

```
[1] 5 5 1
```

```
# Rank-1 approx
X_rank1 <- D[1] * U[, 1] %*% t(V[, 1])

# frobenius norm of the difference
frob_norm <- norm(X - X_rank1, type = "F")
frob_norm
```

```
[1] 5.09902
```

```r
# double check
sqrt(D[2]^2 + D[3]^2)
```

```
[1] 5.09902
```

b) **QR and Least Squares** (6 points): Using the data below, create the design matrix, solve the least squares problem via QR decomposition, compare with `lm()`, and explain the advantages of QR decomposition for least squares.

QR decomposition is better for least squares because it's more stable than using the normal equation approach. The normal equation uses for the formula beta = (t(X) %% *X)^(-1)* %% t(X) %% *y, which requires forming t(X)* %% X then inverting it. The problem is that this step makes the calculation more sensitive to small rounding errors, especially when X are similar to each other, which can effect the coefficient estimates. QR avoids this problem by working directly on X, then once we have the upper triangular R matrix we can solve it by using back substitution, which is why lm() uses QR as well.

```r
set.seed(228)
n <- 100
x <- runif(n, -5, 5)
y <- 3 + 2 * x + rnorm(n, sd = 1)

X <- cbind(1, x) # col of 1's for intercept, predictor x (slope)
qr_X <- qr(X)
beta_qr <- qr.coef(qr_X, y)
beta_qr
```

```
       x
2.985095 1.950611
```

```r
fit_lm <- lm(y ~ x)
beta_lm <- coef(fit_lm)
beta_lm
```

```
(Intercept)           x
   2.985095    1.950611
```

Part 3: Optimization and Simulation (40 points)

3.1 Gradient Descent (15 points)

Minimize the Rosenbrock function: f(x,y) = (1-x)^2 + 100(y-x^2)2

a) (3 points) Write `rosenbrock(x)` that takes a vector x = c(x1, x2) and returns the function value.

```
# x1: x coord
# x2: y coord
rosenbrock <- function(x){
  x1 <- x[1]
  x2 <- x[2]
  (1-x1)^2 + 100 * (x2 - x1^2)^2
}

# testing
#rosenbrock(c(1,1))
```

b) (5 points) Write `rosenbrock_grad(x)` that returns the gradient vector.

```
rosenbrock_grad <- function(x){
  x1 <- x[1]
  x2 <- x[2]

  dfdx1 <- -2 * (1-x1) - 400 * x1 * (x2 - x1^2) # partial with respect to x
  dfdx2 <- 200 * (x2 - x1^2) # partial with respect to y

  c(dfdx1, dfdx2)
}

# testing
rosenbrock_grad(c(0,0))
```

```
[1] -2  0
```

c) (4 points) Implement `gradient_descent(init, grad_fn, learning_rate, max_iter, tol)` that returns the final solution, function value, and iteration count. Stop when the Euclidean norm of the gradient is less than the tolerance.

```
gradient_descent <- function(init, grad_fn, learning_rate, max_iter, tol){
  x <- init # initialize at zero

  for (i in 1:max_iter){
```

```
    # compute gradient
    gradient <- grad_fn(x)

    # check convergence, stop if gradient norm below tol
    if(sqrt(sum(gradient^2)) < tol){
      break
    }
    # update x
    x <- x - learning_rate * gradient
  }
  return(list(
    solution = x,
    value = rosenbrock(x),
    iterations = i
  ))
}
```

d) (3 points) Apply from starting point (-1.2, 1.0) with learning_rate = 0.001, max_iter = 10000, tol = 1e-6. Report the solution, function value, and iterations.

```
result <- gradient_descent(
  init = c(-1.2, 1.0),
  grad_fn = rosenbrock_grad,
  learning_rate = 0.001,
  max_iter = 10000,
  tol = 1e-6
)

result
```

```
$solution
[1] 0.9923558 0.9847394

$value
[1] 5.852761e-05

$iterations
[1] 10000
```

3.2 Comparison with Built-in Methods (10 points)

a) (5 points) Use `optim()` with "BFGS" to minimize the Rosenbrock function from the same starting point.

```r
result_bfgs <- optim(
  par = c(-1.2, 1.0),
  fn = rosenbrock,
  gr = rosenbrock_grad,
  method = "BFGS"
)

result_bfgs
```

```
$par
[1] 1 1

$value
[1] 9.594955e-18

$counts
function gradient
     110       43

$convergence
[1] 0

$message
NULL
```

b) (5 points) Use `optim()` with "Nelder-Mead". Compare all three methods (your gradient descent, BFGS, Nelder-Mead) in terms of solution accuracy, iterations, and function evaluations.

BFGS was the most accurate and most efficient, getting the exact minimum (1,1) with a function value of about 9.6e-18 in 43 gradient evaluations. Nelder-Mead got close at (1.0003, 1.0005) with a function value of 8.8e-08 using 195 function evaluations and no gradient evaluations since it does not use derivatives. Gradient descent was the least accurate, ending at (0.992, 0.985) with a function value of 5.85e-05 after running 10,000 iterations without converging. BFGS performed the best because it uses both the gradient and an estimate of the curvature to decide how to step. Gradient descent uses a fixed size step, but in BFGS it adapts size and direction of each step based on the shape of the function. Nelder-Mead got second place because it doesn't use the gradient at all, so it has less information to work with than BFGS.

```r
result_nm <- optim(
  par = c(-1.2, 1.0),
  fn = rosenbrock,
```

```
  method = "Nelder-Mead"
)

result_nm
```

```
$par
[1] 1.000260 1.000506

$value
[1] 8.825241e-08

$counts
function gradient
     195       NA

$convergence
[1] 0

$message
NULL
```

3.3 Simulation Study (15 points)

Compare the mean, median, and 10% trimmed mean as estimators of central tendency.

a) (7 points) Write `run_simulation(n_samples, n_reps, dist_type)` that generates `n_reps` samples of size `n_samples`, computes all three estimators, and returns a data frame.

```
run_simulation <- function(n_samples, n_reps, dist_type){
  estimates <- matrix(NA,
                      nrow = n_reps,
                      ncol = 3,
                      dimnames = list(NULL, c("mean", "median", "trimmed")))

  for (k in seq_len(n_reps)){
    if(dist_type == "normal"){
      x <- rnorm(n_samples, mean = 0, sd = 1)
    } else if (dist_type == "contaminated"){
      n_clean <- rbinom(1, n_samples, 0.9)
      x_clean <- rnorm(n_clean, mean = 0, sd = 1)
      x_cont  <- rnorm(n_samples - n_clean, mean = 0, sd = 10)
```

```
      x <- c(x_clean, x_cont)
    } else if (dist_type == "t"){
      x <- rt(n_samples, df = 3)
    } else {
      stop("dist type must be either normal, contaminated, or t")
    }
    estimates[k, "mean"] <- mean(x)
    estimates[k, "median"] <- median(x)
    estimates[k, "trimmed"] <- mean(x, trim = 0.1)

  }
  as.data.frame(estimates)
}
```

b) (4 points) Run with n = 30, 1000 replications, under three distributions: normal(0,1), t(df=3), and contaminated normal (90% from N(0,1), 10% from N(0,10)). Calculate bias, variance, and MSE for each estimator under each distribution.

```
set.seed(228)
results_normal <- run_simulation(30, 1000, "normal")
results_t <- run_simulation(30, 1000, "t")
results_contaminated <- run_simulation(30, 1000, "contaminated")

#results_normal
#results_t
#results_contaminated

bias_normal <- colMeans(results_normal)
bias_t <- colMeans(results_t)
bias_contaminated <- colMeans(results_contaminated)

var_normal <- apply(results_normal, 2, var)
var_t <- apply(results_t, 2, var)
var_contaminated <- apply(results_contaminated, 2, var)

mse_normal <- colMeans(results_normal^2)
mse_t <- colMeans(results_t^2)
mse_contaminated <- colMeans(results_contaminated^2)

cat("Normal(0, 1):\n")
```

```
Normal(0, 1):
```

```
print(rbind(bias = bias_normal, variance = var_normal, mse = mse_normal))
```

```
                mean     median     trimmed
bias     0.002411002 0.01089674 0.002901703
variance 0.031730758 0.04558425 0.032611863
mse      0.031704841 0.04565740 0.032587671
```

```
cat("\nt(df = 3):\n")
```

```
t(df = 3):
```

```
print(rbind(bias = bias_t, variance = var_t, mse = mse_t))
```

```
                mean      median     trimmed
bias     -0.02597973 -0.01481375 -0.01592796
variance  0.10480663  0.05757230  0.05664249
mse       0.10537677  0.05773418  0.05683955
```

```
cat("\nContaminated:\n")
```

```
Contaminated:
```

```
print(rbind(bias = bias_contaminated, variance = var_contaminated, mse = mse_contaminate
```

```
                mean     median     trimmed
bias     0.008108931 0.00451098 0.007898796
variance 0.344576870 0.06055395 0.055926903
mse      0.344298048 0.06051374 0.055933367
```

c) (4 points) Create a summary table and write a brief paragraph (3-5 sentences) interpreting which estimator performs best under which conditions and why.

Under the normal distribution N(0,1) the mean estimator performed with MSE 0.032 because the mean uses all the data with equal weight and when there are no outliers that gives the lowest variance. For the t(df = 3) distribution, trimmed mean performed the best with an MSE of 0.057, since heavy tails produce extreme values that pull the mean around but get cut off by trimming. For the contamination distribution, trimmed mean performed the best with an MSE of 0.056 and that's because the outliers from N(0,10) throws off the average. The median is also robust, but it only depends on the middle value, but it throws away most of the data, which is why its MSE is slightly higher than the trimmed mean.

```r
summary_table <- data.frame(
  distribution = rep(c("Normal", "t(df=3)", "Contaminated"), each = 3),
  estimator = rep(c("mean", "median", "trimmed"), times = 3),
  bias = c(bias_normal, bias_t, bias_contaminated),
  variance = c(var_normal, var_t, var_contaminated),
  mse = c(mse_normal, mse_t, mse_contaminated)
)

summary_table
```

```
  distribution estimator         bias   variance        mse
1       Normal      mean  0.002411002 0.03173076 0.03170484
2       Normal    median  0.010896738 0.04558425 0.04565740
3       Normal   trimmed  0.002901703 0.03261186 0.03258767
4      t(df=3)      mean -0.025979726 0.10480663 0.10537677
5      t(df=3)    median -0.014813751 0.05757230 0.05773418
6      t(df=3)   trimmed -0.015927960 0.05664249 0.05683955
7 Contaminated      mean  0.008108931 0.34457687 0.34429805
8 Contaminated    median  0.004510980 0.06055395 0.06051374
9 Contaminated   trimmed  0.007898796 0.05592690 0.05593337
```

Part 4: Bootstrap Methods (15 points)

4.1 Bootstrap Confidence Intervals (8 points)

a) (4 points) Implement `bootstrap_mean()` that estimates the standard error and 95% confidence interval for the mean using the nonparametric bootstrap. Support three CI types: "percentile", "basic", and "normal". Apply to the returns data below and report all three intervals.

```r
library(boot)

bootstrap_mean <- function(data, n_boot = 10000, ci_type = "percentile", conf_level = 0.
  sample_mean <- function(data, indices){
    mean(data[indices])
  }

  boot_result <- boot(data = data, statistic = sample_mean, R = n_boot)
  se_boot <- sd(boot_result$t)
```

```
  type_map <- c(percentile = "perc", basic = "basic", normal = "norm")
  boot_type <- type_map[[ci_type]]

  ci_result <- boot.ci(boot_result, conf = conf_level, type = boot_type)
  if (ci_type == "percentile"){
    ci <- ci_result$percent[4:5]
  } else if (ci_type == "basic"){
    ci <- ci_result$basic[4:5]
  } else {
    ci <- ci_result$normal[2:3]
  }

  list(
    estimate = boot_result$t0,
    se = se_boot,
    ci = ci,
    ci_type = ci_type,
    boot_means = boot_result$t
  )

}
```

```
set.seed(42)
returns <- rnorm(50, mean = 0.08, sd = 0.12)
returns <- returns + rexp(50, rate = 20) - 0.05

set.seed(228); result_pct <- bootstrap_mean(returns, ci_type = "percentile")
set.seed(228); result_basic <- bootstrap_mean(returns, ci_type = "basic")
set.seed(228); result_normal <- bootstrap_mean(returns, ci_type = "normal")

cat("Sample mean:", round(result_pct$estimate, 4), "\n")
```

```
Sample mean: 0.0825
```

```
cat("Bootstrap SE:", round(result_pct$se, 4), "\n")
```

```
Bootstrap SE: 0.0217
```

```
cat("Percentile CI:", round(result_pct$ci, 4), "\n")
```

```
Percentile CI: 0.0396 0.1253
```

```
cat("Basic CI:", round(result_basic$ci, 4), "\n")
```

```
Basic CI: 0.0397 0.1255
```

```
cat("Normal CI:", round(result_normal$ci, 4), "\n")
```

```
Normal CI: 0.0399 0.1252
```

b) (4 points) Create a histogram of the bootstrap distribution of the mean with vertical lines indicating the three different confidence intervals. Discuss which CI method is most appropriate for this data and why (3-5 sentences).

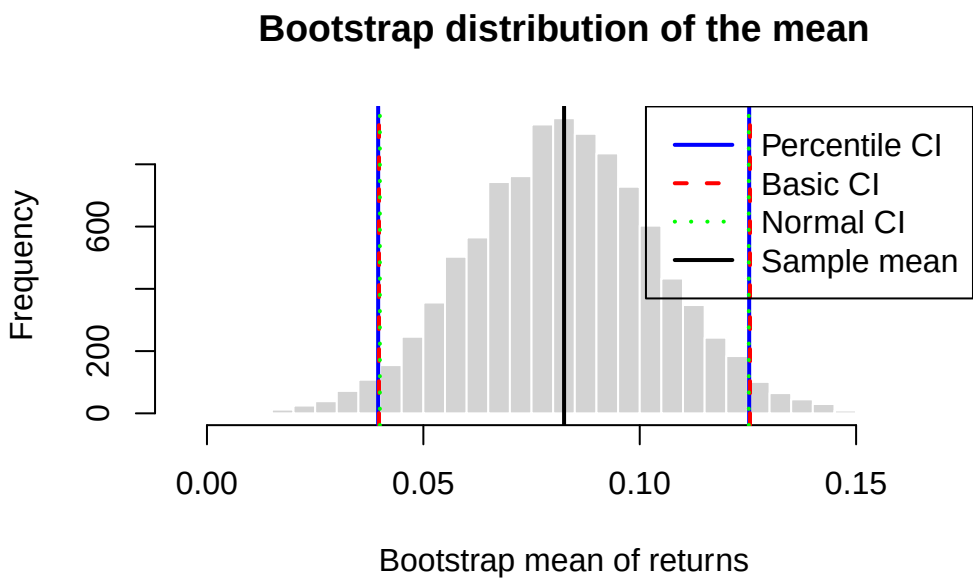
The bootstrap distribution of the mean looks symmetric and bell shaped. The three confidence intervals are almost identical about [0.04, 0.125]. This is expected for a sample size of 50 because the Central Limit Theorem makes the sampling distribution of the mean approximately normal. Since the distribution is symmetric, all three methods are appropriate, but the percentile CI is the safest choice because it does not assume normality and works for any shape of bootstrap distribution.

```
boot_means <- result_pct$boot_means

hist(boot_means,
     breaks = 40,
     main   = "Bootstrap distribution of the mean",
     xlab   = "Bootstrap mean of returns",
     col    = "lightgray",
     border = "white")

abline(v = result_pct$ci, col = "blue",  lwd = 2, lty = 1)
abline(v = result_basic$ci, col = "red",   lwd = 2, lty = 2)
abline(v = result_normal$ci, col = "green", lwd = 2, lty = 3)
abline(v = mean(returns), col = "black", lwd = 2, lty = 1)

legend("topright",
       legend = c("Percentile CI", "Basic CI", "Normal CI", "Sample mean"),
       col = c("blue", "red", "green", "black"),
       lty = c(1, 2, 3, 1),
       lwd = 2)
```



4.2 Bootstrap Hypothesis Testing (7 points)

a) (4 points) Implement `bootstrap_ttest()` that performs a bootstrap test for the difference in means between two groups. Apply to the data below with 2000 bootstrap samples. Create a visualization showing the bootstrap null distribution with the observed difference marked.

```
bootstrap_ttest <- function(group1, group2, n_boot = 2000){
  obs_diff <- mean(group2) - mean(group1)

  pooled_mean <- mean(c(group1, group2))
  group1_shifted <- group1 - mean(group1) + pooled_mean
  group2_shifted <- group2 - mean(group2) + pooled_mean

  boot_diffs <- numeric(n_boot)
  for (b in seq_len(n_boot)){
    boot_g1 <- sample(group1_shifted, length(group1_shifted), replace = TRUE)
    boot_g2 <- sample(group2_shifted, length(group2_shifted), replace = TRUE)
    boot_diffs[b] <- mean(boot_g2) - mean(boot_g1)
  }

  p_val <- mean(abs(boot_diffs) >= abs(obs_diff))

  list(
    obs_diff = obs_diff,
```

```
    p_val = p_val,
    boot_diffs = boot_diffs
  )
}
```

```
set.seed(123)
control <- rnorm(40, mean = 10, sd = 2)
treatment <- rnorm(40, mean = 11.5, sd = 2)

set.seed(228)
result <- bootstrap_ttest(control, treatment, n_boot = 2000)
cat("Observed difference:", round(result$obs_diff, 4), "\n")
```

```
Observed difference: 1.3962
```

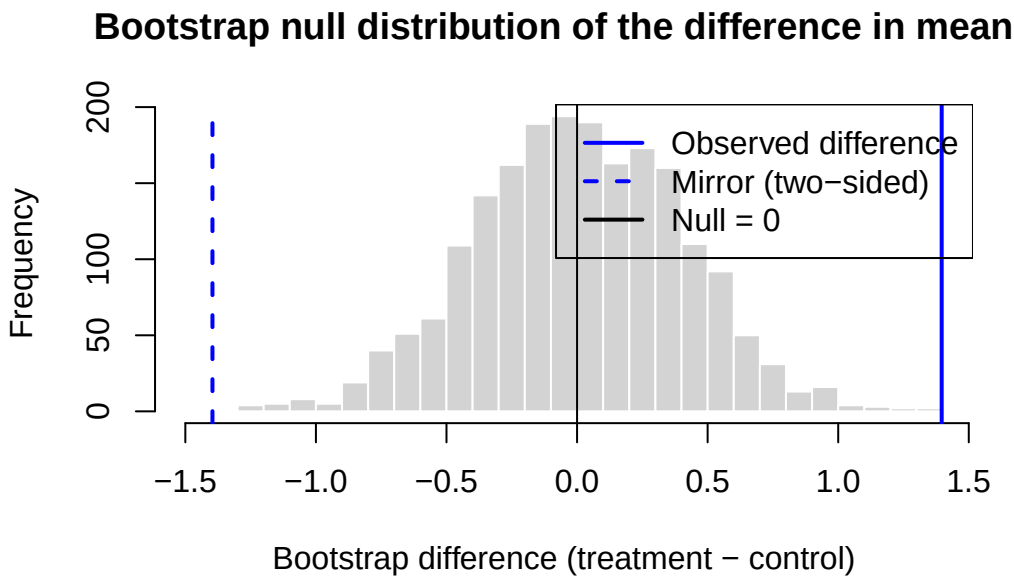
```
cat("Bootstrap p-value:", round(result$p_val, 4), "\n")
```

```
Bootstrap p-value: 5e-04
```

```
hist(result$boot_diffs,
     breaks = 40,
     main   = "Bootstrap null distribution of the difference in means",
     xlab   = "Bootstrap difference (treatment - control)",
     col    = "lightgray",
     border = "white")

abline(v = result$obs_diff,  col = "blue",   lwd = 2)
abline(v = -result$obs_diff, col = "blue",   lwd = 2, lty = 2)
abline(v = 0, col = "black", lwd = 1)

legend("topright",
       legend = c("Observed difference", "Mirror (two-sided)", "Null = 0"),
       col = c("blue", "blue", "black"),
       lty = c(1, 2, 1),
       lwd = 2)
```



b) (3 points) Compare your bootstrap test results with a standard t-test. Discuss any differences and potential advantages of the bootstrap approach (3-5 sentences).

Both methods gave small p values and agree that the difference between the treatment and control groups is statistically significant (bootstrap: 0.0005, t test: 0.0012). t test relies on the assumption that the data are approximately normally distributed, while the bootstrap does not assume any specific distribution and instead uses re sampling to build the null distribution directly from the data. This makes bootstrap approach more flexible since it works even when the data is skewed, outliers, or from unknown distribution. The histogram above makes it easy to see, since the observed difference of 1.396 is far out in the tail and no bootstrap differences come close to it, which visually confirms the very small p value.

```
t_result <- t.test(treatment, control)
t_result
```

```
    Welch Two Sample t-test

data:  treatment and control
t = 3.3597, df = 77.656, p-value = 0.001212
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 0.5688065 2.2235966
sample estimates:
mean of x mean of y
 11.48657  10.09037
```

```
result$p_val
```

```
[1] 5e-04
```

```
result$obs_diff
```

```
[1] 1.396202
```